

# ClaimFlow (Receipt & Employee Claims Management)

## Core Features

For Employees:

- **Receipt Uploads:** Securely upload image files (JPG, PNG) or PDFs of physical receipts.
- **Claim Submission:** Create an expense claim by linking uploaded receipts, categorizing the expense (e.g., Travel, Meals, Supplies), and inputting the total amount.
- **Status Tracking:** View a real-time dashboard of all submitted claims and their current status (Pending, Approved, Rejected, Processed).

For Managers / Admins:

- **Approval Workflow:** Dashboard displaying a queue of pending claims submitted by team members.
- **Review Mechanics:** Ability to view the attached receipt image side-by-side with the claimed amount to verify accuracy.
- **Action Tracking:** Approve or reject claims with the mandatory option to leave a comment/reason for rejections.

Database Schema (High-Level)

- **Users:** ID, Name, Email, Role (Employee/Admin), Department\_ID.
- **Claims:** ID, User\_ID, Total\_Amount, Status, Submission\_Date, Admin\_Notes.
- **Receipts:** ID, Claim\_ID, Image\_URL, Merchant\_Name, Expense\_Date, Category.

## Phase 1: User Auth & Registration

### 1. Account Creation:

- Employee registers via the React frontend.
- FastAPI receives plaintext password, computes `hashed_password = pwd_context.hash(password)`, and saves the document in the `users` MongoDB collection.

### 2. Login & Verification:

- User submits email and password.
- FastAPI fetches the user document by email and verifies `pwd_context.verify(submitted_password, user.password_hash)`.
- On match, FastAPI issues an auth session token or simple JWT.

## Phase 2: Claim Submission & Local File Upload

### 1. Form Completion:

- Employee enters manual claim details on the frontend: **Amount**, **Merchant**, **Category**, **Expense Date**, and selects the receipt image/PDF.

### 2. Multipart Form Upload:

- Frontend sends a single `multipart/form-data` POST request to `/api/v1/claims` containing both the form fields and the uploaded file.

### 3. Local File Handling (FastAPI):

- FastAPI generates a unique filename (e.g., `uuid4() + extension`).
- Reads the file stream and writes it directly to disk at `./uploads/<unique_filename>`.

### 4. Database Insertion:

- FastAPI constructs the receipt document containing the local path and public static URL (`http://backend-domain/uploads/<unique_filename>`).
- Inserts the complete claim document into MongoDB's `claims` collection with `status: "PENDING"`.

## Phase 3: Admin Review Flow

### 1. Fetch Queue:

- Admin dashboard queries `GET /api/v1/admin/claims?status=PENDING`.
- FastAPI queries MongoDB: `db.claims.find({"status": "PENDING"})` and returns the list.

### 2. Review & Display:

- The React UI renders claim details alongside the image tag pointing to `http://backend-domain/uploads/<unique_filename>` (served directly by FastAPI `StaticFiles`).

### 3. Status Update:

- Admin clicks **Approve** or **Reject**.
- Frontend sends `PATCH /api/v1/admin/claims/{claim_id}` with `{ "status": "APPROVED", "admin_notes": "Verified" }`.
- FastAPI performs `db.claims.update_one({"_id": ObjectId(claim_id)}, {"$set": {...}})`.

## Phase 4: Local Setup & Deployment

### 1. FastAPI Static File Serving:

#### 2. Python

- `from fastapi import FastAPI`
- `from fastapi.staticfiles import StaticFiles`
- 
- `app = FastAPI()`
- `app.mount("/uploads", StaticFiles(directory="uploads"), name="uploads")`

### 3. Dockerization:

- `Dockerfile` creates the `/app/uploads` directory inside the container.
- `docker-compose.yml` mounts a host volume or docker volume to `/app/uploads` so files persist across updates.

### 4. Deployment Hosting:

- **Frontend:** Pushed to GitHub and auto-deployed on Vercel or Netlify.
- **Backend:** Docker image deployed to Render or Railway with a **Persistent Volume** attached to the `/uploads` path.
- **Database:** Hosted on MongoDB Atlas (free/pay-as-you-go managed tier) or running as a containerized service.